The lecture hall quadratic assignment problem

Tal Raviv

April 2026

1 Introduction and background

Universities and colleges often determine the courses timetable well before they assign lectures to specific halls. Once the timetable is fixed and student registration is completed, the institution knows both the size of each lecture and, to some extent, the overlap in enrollment between consecutive lectures. At that stage, the hall-assignment decision can have a substantial effect on student mobility across campus. Assigning consecutive lectures attended by many of the same students to distant halls may generate unnecessary walking, even when every lecture is assigned to a feasible room. In practice, long walks between halls may also cause students to arrive late at their next class, thereby disrupting the beginning of lecture. It may also be particularly challenging to students with mobility disabilities.

This observation motivates a lecture-hall assignment problem that is separate from the timetabling itself. In the lecture-hall assignment problem, the timetable, including the scheduled time interval of each lecture, is given. The remaining task is to assign lectures to halls in a way that respects room capacities and temporal compatibility, while also reducing the walking burden imposed on students as they move between consecutive classes. The proposed model is related to three main streams of literature: the quadratic assignment problem (QAP), classroom assignment and interval scheduling, and university course timetabling. The closest conceptual link to the QAP is the presence of pairwise interaction costs: assigning two lectures to two halls creates a cost that depends jointly on the distance between the halls and on the number of students who attend both lectures. This is analogous to the flow–distance structure that defines the classical QAP introduced by Koopmans and Beckmann (1957) and surveyed extensively by Burkard et al. (1998) and Loiola et al. (2007). The QAP literature has developed a rich toolkit of solution techniques whose ideas we draw on in this paper. Classical lower bounds begin with the Gilmore–Lawler bound (Gilmore, 1962; Lawler, 1963), which is obtained from a linear assignment relaxation of the quadratic cost structure. Linearization-based formulations were proposed by Kaufman and Broeckx (1978), who introduced a compact Benders-style reformulation using one auxiliary continuous variable per pair of facilities, and by Frieze and Yadegar (1983), whose linearization exploits the permutation structure. Adams and Johnson (1994) later developed a stronger mixed-integer linear reformulation whose linear-programming relaxation dominates earlier bounds, at the cost of a larger variable set. More recent advances include semidefinite relaxations (Zhao et al., 1998), convex quadratic reformulations and improved branch-and-bound methods (Anstreicher, 2003), and large-scale computational studies such as Anstreicher et al. (2002), who solved previously intractable QAPLIB

instances on computational grids. On the heuristic side, robust tabu search (Taillard, 1991) and related metaheuristics remain the dominant approach for obtaining high-quality solutions to large QAP instances. The compact linearization and the valid inequalities that we develop in Section 3 are inspired by this line of work but are adapted to the combined temporal and compatibility structure of the lecture-hall setting. At the same time, the present problem differs from the standard QAP because feasibility is driven by temporal overlap, room capacities, and the subset of lecture pairs that are consecutive on the same day.

A second relevant stream concerns classroom assignment and interval-based room allocation. In this family of problems, activities with fixed or partially fixed time intervals must be assigned to rooms while respecting incompatibilities caused by overlap in time. Carter and Tovey (1992) study the classroom assignment problem from a complexity perspective and explicitly connect it to interval scheduling. The broader interval-scheduling literature, surveyed by Kolen et al. (2007), studies machine or resource assignment of activities with fixed time windows and provides a natural framework for the temporal-feasibility layer of our problem. In particular, the clique structure of interval graphs, which we exploit in Section 3 to tighten the overlap constraints, is a standard building block in that literature. The framework of Carter and Tovey (1992) and Kolen et al. (2007) is closely related to the feasibility component of the present model, where lectures cannot share a hall if their time intervals overlap. However, the objective considered here is richer because it is not limited to feasibility or room utilization; instead, it introduces interaction costs across consecutive lectures through student flows and hall-to-hall distances.

A third stream is the university timetabling literature, which studies the joint allocation of courses to times and rooms under institutional constraints. General surveys by Schaerf (1999) and Burke and Petrovic (2002) describe a broad range of formulations, including problems in which room capacities, conflicts, and soft preferences play a central role, while the more recent review of Abdipoor et al. (2023) summarizes modern meta-heuristic and hybrid approaches for university course timetabling. Within this literature, classroom assignment is often treated either as a subproblem of course timetabling or as a second-stage problem after times have been fixed. For example, Torres-Ovalle et al. (2014) propose a two-phase approach in which course scheduling is followed by classroom assignment, and Rappos et al. (2022) develop a mixed-integer programming model that jointly considers student allocation, room assignment, and time-period assignment. A recent extension by Davison et al. (2025) brings hybrid teaching into the university course timetabling problem by treating each class's delivery mode: fully online, fully in-person, or hybrid with a split of students between the two modes, as a first-class modelling element alongside the usual room and time decisions. They formulate a multi-objective binary program that combines standard timetabling features identified from a broad literature review with new variables and constraints describing in-person versus online delivery and student mode preferences, and they solve it by a lexicographic method on benchmark data, illustrating trade-offs among demand satisfaction, room utilisation, and hybrid-delivery quality for use in strategic decision making. Their work expands the set of constraints and objectives that a timetabling model is expected to handle but, like most models in this tradition, keeps room assignment embedded inside the larger multi-criterion formulation rather than isolating it as a separate problem with exploitable structure. This decomposition is natural in many university settings and is also compatible with the present formulation, which assumes that the scheduled time interval of each lecture is already known.

Taken together, these three literatures suggest that the lecture hall assignment problem studied here combines structural ingredients that are usually treated separately. From classroom assignment and

interval scheduling it inherits the temporal packing constraints on rooms; from timetabling it inherits the academic context, capacity restrictions, and interpretation of lectures as scheduled activities; and from the QAP it inherits a pairwise objective that couples assignment decisions across lectures. A particularly close recent connection is the room-allocation study of Antunes-Batista et al. (2026), who isolate the room-assignment subproblem of the university course timetabling problem under an already-fixed weekly schedule and formulate it as a six-criterion problem with two hard constraints (room capacity and no scheduling overlap) and four soft objectives (student relocation between buildings, mismatch between requested and assigned room types, wasted capacity, and room changes between consecutive classes of the same student group). They develop customised multiobjective evolutionary algorithms augmented with problem-specific local search and constructive heuristics in the jMetal framework, and benchmark them against an ILP model implemented in Pyomo and solved by Gurobi. The computational study uses a full-semester dataset from a real university with approximately 10,000 students, more than 26,000 scheduled class sessions, and over 120 classrooms of more than 20 distinct types, and emphasises scalability and Pareto-front quality as the central performance measures.

The present framework can likewise accommodate mandatory room features through the compatibility sets and additional room-quality criteria through linear assignment penalties, so on the modelling surface it overlaps with four of the six criteria of Antunes-Batista et al. (2026). Its distinguishing features are structural and methodological. Structurally, student movement enters as a QAP-style quadratic term induced by shared enrollments and inter-hall walking distances rather than as a count of building changes or group-level room changes; capacity and overlap are retained as hard feasibility constraints; and the remaining room-quality criteria are combined into a single scalar objective through linear assignment penalties, which is possible because the weighting between them is under the analyst's control. Methodologically, this combined structure motivates exact mathematical-programming and constraint-programming approaches with problem-specific valid inequalities and preprocessing, rather than the multiobjective evolutionary framework favoured in Antunes-Batista et al. (2026). The two approaches are therefore complementary: theirs targets very large operational instances where a Pareto front among competing criteria is the primary deliverable, while ours targets a clean QAP-flavoured subproblem where exploiting pairwise structure and interval-graph cliques yields provably optimal or near-optimal solutions.

Research gap and contribution. The three streams reviewed above leave a clear gap. The QAP literature develops powerful linearizations, bounds, and exact algorithms (Kaufman and Broeckx, 1978; Frieze and Yadegar, 1983; Adams and Johnson, 1994; Zhao et al., 1998; Anstreicher et al., 2002; Anstreicher, 2003), but it does so for the pure permutation problem with unrestricted assignments; it does not natively handle heterogeneous room capacities, temporal overlap, or partial compatibility. The interval-scheduling literature (Carter and Tovey, 1992; Kolen et al., 2007) provides the right feasibility primitives but typically works with linear or feasibility-only objectives. The university timetabling and room-allocation literature accounts for room capacities, time conflicts, and in some cases student-relocation objectives, but these decisions are usually embedded in broader timetabling models (Rappos et al., 2022; Davison et al., 2025) or treated as one component of a multiobjective room-allocation problem solved by evolutionary algorithms (Antunes-Batista et al., 2026). In particular, neither the hybrid-teaching binary program of Davison et al. (2025) nor the multiobjective evolutionary room allocator of Antunes-Batista et al. (2026) treats student movement between consecutive classes as a QAP-style quadratic flow–distance term; student re-

location in the latter is counted as building changes or room changes rather than weighted by pairwise walking distances. In contrast, we study a fixed-time lecture-hall assignment problem in which the timetable is given and the dominant structural component is a QAP-style quadratic walking term induced by shared enrollments and inter-hall distances. The framework keeps capacity and overlap as hard constraints, and it can also incorporate mandatory room features through the compatibility sets together with additional linear assignment penalties when desired. This setting is natural in institutions where timetabling decisions are made before registration is finalized and actual course enrollments are then observed. Isolating the hall-assignment layer yields a distinct exact optimization problem with exploitable structure. We analyze its computational complexity, adapt and extend compact QAP-style linearizations to a setting with temporal and capacity constraints, derive mathematical programming and constraint programming formulations together with valid inequalities and safe preprocessing techniques, and show in the numerical study that these enhancements substantially improve computational performance relative to the baseline nonlinear formulation while enabling the solution of large and realistic instances to optimality or near-optimality.

2 Problem statement and complexity

We now formalize the lecture-hall assignment problem and state its complexity properties.

We consider this problem through the following sets:

Halls (H) Each hall is defined by its capacity and location.

Lectures (L) Each lecture is defined by its scheduled time interval and its number of students.

Immediate-successor relation (A) A relation $A \subseteq L \times L$, where $(l_1, l_2) \in A$ if lecture l_2 is scheduled immediately after lecture l_1 for at least one group of students.

The walking distance between each pair of halls $h_1, h_2 \in H$ is given by the distance matrix d_{h_1,h_2}. In addition, for each pair of lectures $(l_1, l_2) \in A$, we are given the number of students enrolled in both classes, denoted by c_{l_1,l_2}. This number may be a simple head count or a score that gives higher weight to students with disabilities, if the institution has this information at its disposal.

The goal is to assign each lecture to a hall with sufficient capacity, such that no two lectures assigned to the same hall overlap in time. The objective is to minimize the total walking distance walked by students between consecutive classes.

This problem shares features with both the quadratic assignment problem and interval scheduling, and in a sense it combines elements of both.

A natural decision version of the lecture-hall assignment problem asks whether, for a given bound B, there exists a feasible hall assignment whose total walking cost is at most B. This decision problem belongs to NP, since a proposed assignment can be verified in polynomial time by checking compatibility, non-overlap, and the resulting objective value. The next proposition states the exact and approximation hardness of the problem.

Proposition 1. *The decision version of the lecture-hall assignment problem is NP-complete, and the corresponding optimization problem is NP-hard. Moreover, for every $\rho \geq 1$, if there exists a polynomial-time ρ-approximation algorithm for the lecture-hall assignment problem, then there exists a polynomial-time ρ-approximation algorithm for the classical quadratic assignment problem (QAP). Consequently, every approximation lower bound for QAP carries over to the lecture-hall assignment problem. All of these statements remain true even for a restricted single-day case with two consecutive time blocks, identical hall capacities, and full lecture-hall compatibility.*

Proof. We use a reduction from the classical QAP, which is NP-hard and inapproximable within a constant factor, see Sahni and Gonzalez (1976); Burkard et al. (1998); Loiola et al. (2007). Consider a QAP instance with facilities $\mathcal{N} = \{1, \ldots, n\}$, locations $\mathcal{R} = \{1, \ldots, n\}$, nonnegative flow matrix (f_{ij}), and nonnegative distance matrix (d_{ab}). First normalize the distances by setting

$$\hat{d}_{aa} = 0 \quad \forall a \in \mathcal{R}, \qquad \hat{d}_{ab} = d_{ab} + 1 \quad \forall a \neq b.$$

For every permutation π of the locations,

$$\sum_{i=1}^{n} \sum_{j=1}^{n} f_{ij} \hat{d}_{\pi(i),\pi(j)} = \sum_{i=1}^{n} \sum_{j=1}^{n} f_{ij} d_{\pi(i),\pi(j)} + \sum_{i \neq j} f_{ij},$$

because $\pi(i) \neq \pi(j)$ whenever $i \neq j$. Thus the normalization adds the same constant to every feasible QAP solution, so it preserves both exact comparisons and approximation ratios.

Now construct a lecture-hall instance as follows. Create n halls $h_1, \ldots, h_n$, all with identical capacity and all compatible with every lecture. Create two sets of lectures,

$$U = \{u_1, \ldots, u_n\} \qquad \text{and} \qquad V = \{v_1, \ldots, v_n\},$$

where all lectures in U occupy one common time block and all lectures in V occupy the immediately following common time block. Every lecture in U therefore overlaps every other lecture in U, and similarly for V. Since there are exactly n halls, every feasible lecture-hall assignment induces two permutations of the halls: one permutation π for the lectures in U and one permutation σ for the lectures in V. For every $i, j \in \{1, \ldots, n\}$, include the pair (u_i, v_j) in the successor set and set

$$d_{h_a,h_b} = \hat{d}_{ab} \qquad \forall a, b \in \{1, \ldots, n\}.$$

Let M be a positive penalty parameter, to be specified below, and define

$$c_{u_i,v_j} = \begin{cases} f_{ij} + M, & \text{if } i = j, \\ f_{ij}, & \text{if } i \neq j. \end{cases}$$

Under a feasible lecture-hall assignment corresponding to permutations (π, σ), the objective value is

$$\sum_{i=1}^{n} \sum_{j=1}^{n} f_{ij} \hat{d}_{\pi(i),\sigma(j)} + M \sum_{i=1}^{n} \hat{d}_{\pi(i),\sigma(i)}.$$

If $\pi \neq \sigma$, then there exists some i such that $\pi(i) \neq \sigma(i)$, and therefore $\hat{d}_{\pi(i),\sigma(i)} \geq 1$. Hence every such assignment has value at least M. On the other hand, if $\pi = \sigma$, then the penalty term vanishes and the lecture-hall objective becomes

$$\sum_{i=1}^{n} \sum_{j=1}^{n} f_{ij} \hat{d}_{\pi(i),\pi(j)},$$

which is exactly the normalized QAP objective.

For the exact reduction, start from the QAP decision problem with threshold K and choose

$$M > \hat{K} := K + \sum_{i \neq j} f_{ij}.$$

Then any lecture-hall solution of value at most $\hat{K}$ must satisfy $\pi = \sigma$, and once $\pi = \sigma$ its value is exactly the normalized QAP value of the same permutation. Therefore the original QAP instance is a yes-instance if and only if the constructed lecture-hall instance admits a feasible assignment of cost at most $\hat{K}$. The reduction is polynomial, so the decision version of the lecture-hall assignment problem is NP-hard. Since it belongs to NP, it is NP-complete. Consequently, the optimization problem is NP-hard.

For the approximation statement, fix $\rho \geq 1$ and let

$$F_{\text{tot}} = \sum_{i=1}^{n} \sum_{j=1}^{n} f_{ij}, \qquad D_{\max} = \max_{a,b \in \mathcal{R}} \hat{d}_{ab}, \qquad U_{\max} = F_{\text{tot}} D_{\max}.$$

Every feasible normalized QAP solution has value at most $U_{\max}$, and therefore the optimal value OPT_{QAP} satisfies $\text{OPT}_{\text{QAP}} \leq U_{\max}$. Now choose

$$M = \rho U_{\max} + 1.$$

Then every lecture-hall solution with $\pi \neq \sigma$ has value at least

$$M = \rho U_{\max} + 1 > \rho U_{\max} \geq \rho \, \text{OPT}_{\text{QAP}}.$$

But the optimal lecture-hall value is exactly OPT_{QAP}, attained by solutions with $\pi = \sigma$. Therefore any polynomial-time ρ-approximation algorithm for the lecture-hall assignment problem, applied to this constructed instance, must return a solution with $\pi = \sigma$, because every solution with $\pi \neq \sigma$ already exceeds ρ times the optimum. For such a returned solution, the lecture-hall objective is exactly the normalized QAP objective of the same permutation. Hence a polynomial-time ρ-approximation for the lecture-hall assignment problem yields a polynomial-time ρ-approximation for QAP.

Thus both exact hardness and approximation hardness follow from the same reduction family. □

3 Models and algorithms

This section presents three complementary exact modeling views of the lecture-hall assignment problem. We begin with a natural nonlinear integer programming formulation, then derive a

compact mixed-integer programming reformulation, and finally present a constraint-programming formulation that captures the assignment structure directly. After these three base formulations, we describe several modeling improvements that can be layered on top of them, including clique-based overlap handling, valid inequalities, bi-clique cuts, and safe compatibility preprocessing.

For notational convenience, define

$$A' = \{(l_1, l_2) \in A : c_{l_1,l_2} > 0\}.$$

3.1 Nonlinear Integer Programming Formulation

A natural nonlinear integer programming formulation is as follows. For each lecture $l \in L$ and each compatible hall $h \in H(l)$, let

$$x_{lh} = \begin{cases} 1 & \text{if lecture } l \text{ is assigned to hall } h, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

Let s_l denote the number of students attending lecture l, and let u_h denote the capacity of hall h. Define $H(l) \subseteq H$ as the set of halls that are compatible with lecture l. In the basic model studied here, compatibility is determined only by capacity, so

$$H(l) = \{h \in H : u_h \geq s_l\}.$$

Fix a day $d \in D$, and define the interval graph $G_d = (L_d, E_d)$ in which two lectures are adjacent if and only if their scheduled intervals overlap. Since lecture intervals lie on a line, G_d is an interval graph. Let $\mathcal{C}_d$ denote the family of maximal cliques of G_d. The problem can then be written as

$$\min \quad \sum_{(l_1,l_2)\in A'} c_{l_1,l_2} \sum_{h_1\in H(l_1)} \sum_{h_2\in H(l_2)} d_{h_1,h_2}\, x_{l_1h_1} x_{l_2h_2} \tag{2}$$

$$\text{s.t.} \quad \sum_{h\in H(l)} x_{lh} = 1, \qquad \forall l \in L, \tag{3}$$

$$\sum_{l\in C:\, h\in H(l)} x_{lh} \leq 1, \qquad \forall d \in D,\ \forall C \in \mathcal{C}_d,\ \forall h \in H, \tag{4}$$

$$x_{lh} \in \{0, 1\}, \qquad \forall l \in L,\ \forall h \in H(l). \tag{5}$$

The objective is quadratic because the cost of assigning a lecture to a hall depends on where the immediately succeeding lecture is assigned. The first set of constraints ensures that each lecture is assigned to exactly one compatible hall, and the second enforces non-overlap by requiring that, for every hall and every maximal clique of mutually overlapping lectures, at most one lecture in that clique may use the hall.

An immediate structural observation is that the problem is separable by day. Let D denote the set of teaching days, and for each $d \in D$ let $L_d \subseteq L$ be the set of lectures scheduled on day d. Likewise, let $A_d \subseteq A$ be the set of successor pairs whose two lectures both belong to day d, and let $T_d \subseteq T$ be the set of time units associated with day d. Because temporal-overlap constraints involve only lectures that occur on the same day, and because an immediate-successor pair must also lie within

a single day, both the objective and feasibility constraints decompose across days. Therefore, the weekly problem can be written as the sum of independent daily subproblems,

$$\min \sum_{d \in D} f_d(x^d), \tag{6}$$

where each f_d is the objective induced by the lectures and successor pairs of day d, and x^d denotes the hall-assignment decisions for that day. In particular, solving the overall weekly problem is equivalent to solving one independent lecture-hall assignment problem for each day and combining the resulting assignments. This observation is useful both computationally and conceptually. Therefore, in what follows one may interpret the model as applying either to the full week or, equivalently, to a single-day instance.

3.2 Mixed-Integer Programming Reformulation

A compact linearization can be obtained by introducing, for each successor pair $(l_1, l_2) \in A$ such that $c_{l_1,l_2} > 0$, one nonnegative continuous variable $z_{l_1 l_2}$ representing the realized hall-to-hall distance induced by that pair. The problem can then be reformulated as the mixed-integer linear program

$$\begin{aligned}
\min \quad & \sum_{(l_1,l_2) \in A'} c_{l_1,l_2} z_{l_1 l_2} && && (7)\\
\text{s.t.} \quad & \text{Constraints (3) and (4)} \\
& z_{l_1 l_2} \geq d_{h_1,h_2}\big(x_{l_1 h_1} + x_{l_2 h_2} - 1\big), && \forall (l_1,l_2) \in A', h_1 \in H(l_1), h_2 \in H(l_2), && (8)\\
& x_{lh} \in \{0,1\}, && \forall l \in L,\ \forall h \in H(l), && (9)\\
& z_{l_1 l_2} \geq 0, && \forall (l_1,l_2) \in A'. && (10)
\end{aligned}$$

This reformulation is exact. Since each lecture is assigned to exactly one hall, for every pair (l_1, l_2) there is exactly one compatible hall pair (h_1, h_2) for which $x_{l_1 h_1} = x_{l_2 h_2} = 1$; for that pair, constraint (8) yields

$$z_{l_1 l_2} \geq d_{h_1,h_2}.$$

For every other hall pair, the right-hand side of (8) is nonpositive. Therefore, because the objective minimizes $\sum c_{l_1,l_2} z_{l_1 l_2}$ and $z_{l_1 l_2}$ is constrained to be nonnegative, the optimal value of $z_{l_1 l_2}$ is exactly the realized distance associated with the halls assigned to l_1 and l_2. Multiplying by c_{l_1,l_2} in the objective recovers the original walking-cost contribution. Restricting the z variables to pairs with $c_{l_1,l_2} > 0$ is sufficient, since pairs with zero shared enrollment do not contribute to the objective.

Compared with the standard disaggregated linearization that introduces one variable for each tuple (l_1, l_2, h_1, h_2), formulation (7)–(10) is substantially more compact in the number of additional variables. In addition, placing the shared-enrollment weight c_{l_1,l_2} in the objective rather than in the linking constraints yields a better-scaled coefficient matrix while preserving the exact same optimal assignment. The room-capacity side is also written in compact form, since the feasibility constraints are imposed only for maximal cliques of the interval graph rather than for every time step separately. However, this compactness does not necessarily imply a stronger linear-programming relaxation on the linking side. Strengthening inequalities for this compact reformulation, and their CP analogues, are described in Subsection 3.4.

3.3 Constraint Programming Formulation

A constraint-programming (CP) formulation can be made more compact by replacing the binary assignment variables with one integer decision variable a_l for each lecture $l \in L$, where a_l denotes the hall assigned to lecture l and its domain is restricted to the compatible set $H(l)$. The model can then be written as

$$\min \quad \sum_{(l_1,l_2)\in A'} c_{l_1,l_2}\, d_{a_{l_1},a_{l_2}} \tag{11}$$

$$\text{s.t.} \quad a_l \in H(l), \qquad \forall l \in L, \tag{12}$$

$$\text{AllDifferent}(\{a_l : l \in L(t)\}), \qquad \forall t \in T. \tag{13}$$

Equation (11) is the CP objective. It minimizes the total walking burden by summing, over all relevant successor pairs, the shared enrollment c_{l_1,l_2} multiplied by the distance between the halls selected by a_{l_1} and a_{l_2}.

Constraint (12) defines the assignment variables. Each lecture is assigned directly to one compatible hall, so the capacity requirement is enforced through the domain of a_l rather than through separate binary variables and assignment equalities.

Constraint (13) enforces temporal feasibility. For every time unit t, all lectures active during that time must take distinct hall values, which ensures that no hall is used simultaneously by two overlapping lectures.

This formulation is a compact high-level CP model. In an implementation based on a CP-SAT solver such as OR-Tools, the hall assignments would still be represented directly by integer variables, and mutual exclusivity could be enforced by `AddAllDifferent`. The objective term $d_{a_{l_1},a_{l_2}}$ may be viewed as concise mathematical shorthand for indexing the distance matrix by decision variables.

If one wants a lower-level encoding that is closer to an actual CP-SAT implementation, one may introduce for each successor pair $(l_1, l_2) \in A$ the same auxiliary integer variable $z_{l_1 l_2}$ used in the MIP reformulation, representing the realized walking distance, together with constraints enforcing

$$z_{l_1 l_2} = d_{a_{l_1},a_{l_2}}. \tag{14}$$

Then the objective in (11) can be replaced by

$$\min \sum_{(l_1,l_2)\in A'} c_{l_1,l_2}\, z_{l_1 l_2}, \tag{15}$$

and the relation (14) can be encoded in CP-SAT by an element constraint or an allowed-assignment table.

3.4 Modeling improvements

Several enhancements can be layered on top of the three base formulations without changing the underlying problem. Some of them are common to all three modeling views, while others are specific to the compact MIP reformulation and its CP analogue.

Clique-based overlap representation. The clique representation introduced in (4) is exact. If one starts instead from slot-indexed constraints, then for every time unit t the active lecture set $L(t)$ forms a clique in the interval graph, and therefore is contained in a maximal clique. Conversely, by the Helly property of intervals, every clique of an interval graph has a common intersection point, so each maximal clique corresponds to a set of lectures that are simultaneously active at some time. Hence keeping only the maximal active sets is equivalent to the full family of slot-indexed non-overlap constraints. In many practical timetables, and especially in discretized slot-based data such as the instances generated here, it is very common that each time step induces a unique maximal active set. In that case the maximal cliques are particularly easy to identify: one simply scans the time steps, records the active lectures at each step, and removes duplicate or inclusion-wise dominated sets. Even when several consecutive time steps map to the same maximal clique, this scan-based identification remains trivial. In the nonlinear and compact MIP formulations, writing the room-capacity constraints by maximal cliques eliminates redundant slot-indexed inequalities without weakening the model. In the CP formulation, one may either post `AllDifferent` on the slot-active sets $L(t)$ or on maximal cliques; the latter is strongest in general, while in slot-based generated instances the two views are typically nearly identical in practice because each slot usually induces a unique maximal active set.

Capacity-dominance cuts. A first general improvement combines the room-capacity structure with the clique-overlap restriction. For each day $d \in D$, each maximal clique $C \in \mathcal{C}_d$, and each capacity threshold $\kappa \geq 0$, define the sets

$$L_\kappa(C) = \{\, l \in C : s_l > \kappa \,\}, \qquad H_\kappa = \{\, h \in H : u_h > \kappa \,\}.$$

Every lecture $l \in L_\kappa(C)$ has $H(l) \subseteq H_\kappa$ by the capacity definition, so all of its assignment mass is placed on halls in H_κ. Summing the clique-overlap inequalities (4) over $h \in H_\kappa$ and isolating the contribution of the large lectures yields the *capacity-dominance cut*

$$\sum_{l \in C \setminus L_\kappa(C)} \; \sum_{h \in H(l) \cap H_\kappa} x_{lh} \;\leq\; |H_\kappa| - |L_\kappa(C)|, \qquad \forall d \in D,\ \forall C \in \mathcal{C}_d,\ \forall \kappa \geq 0. \tag{16}$$

Because $|L_\kappa(C)|$ and $|H_\kappa|$ depend only on the input data, the cut is an explicit linear inequality in the assignment variables whose right-hand side is a pre-computed nonnegative integer. It states that once the $|L_\kappa(C)|$ largest lectures of the clique have claimed their share of the $|H_\kappa|$ halls whose capacity exceeds κ, the remaining lectures in C may collectively occupy at most $|H_\kappa| - |L_\kappa(C)|$ of the large halls. Only threshold values κ that coincide with distinct hall capacities need to be instantiated, so the family contains at most $|\mathcal{C}_d| \cdot |\{u_h : h \in H\}|$ inequalities per day. The cut is a Chvátal–Gomory consequence of (3)–(4) and is therefore valid for every integer-feasible solution of the nonlinear, MIP, and CP formulations alike. In the nonlinear and compact MIP formulations it can be added directly as a linear inequality in the assignment variables. In the CP formulation it can be represented by Boolean indicators that record whether a lecture is assigned to a hall in H_κ, followed by the corresponding redundant cardinality bound. If preprocessing is applied beforehand, the cut can be posted on the reduced compatibility sets without modification.

Bi-clique cuts. The next family of improvements is specific to the compact MIP reformulation. Fix a pair $(l_1, l_2) \in A'$ and a pair of halls $h_1 \in H(l_1)$ and $h_2 \in H(l_2)$, let $S_1 \subseteq H(l_1)$ and $S_2 \subseteq H(l_2)$

such that $d_{h'_1,h'_2} \geq d_{h_1,h_2}$ for every $h'_1 \in S_1$ and $h'_2 \in S_2$. Then the following inequality is valid:

$$z_{l_1 l_2} \geq d_{h_1,h_2} \left(\sum_{h'_1 \in S_1} x_{l_1 h'_1} + \sum_{h'_2 \in S_2} x_{l_2 h'_2} - 1 \right), \qquad \forall (l_1, l_2) \in A', h_1 \in H(l_1), h_2 \in H(l_2). \tag{17}$$

If the assigned hall of l_1 lies in S_1 and that of l_2 lies in S_2, then both summations equal 1 and (17) forces $z_{l_1 l_2} \geq d_{h_1,h_2}$, which is correct since the realized distance is at least d_{h_1,h_2}. Otherwise the right-hand side is nonpositive and the cut is implied by the nonnegativity of $z_{l_1 l_2}$. The basic constraints (8) are the special case $S_1 = \{h_1\}$ and $S_2 = \{h_2\}$; taking larger bi-cliques yields strictly stronger cuts that can separate fractional solutions satisfying all pairwise constraints. In practice, the sets S_1 and S_2 are constructed by fixing an anchor pair $(h_1, h_2) \in H(l_1) \times H(l_2)$, setting $S_2 = \{h \in H(l_2) : d_{h_1,h} \geq d_{h_1,h_2}\}$, and then setting S_1 to every hall in $H(l_1)$ at distance at least d_{h_1,h_2} from every element of S_2. In particular $h_1 \in S_1$ and $h_2 \in S_2$. When the lower-level CP encoding with auxiliary variables $z_{l_1 l_2}$ is used, the same logic can be translated into propagation constraints enforcing lower bounds on $z_{l_1 l_2}$ whenever the two assigned halls fall into the corresponding threshold sets.

Compatibility preprocessing. A final improvement shrinks the compatibility sets $H(l)$ before solving any of the three formulations. Fix a target lecture $l' \in L$ and consider the auxiliary problem

$$\begin{aligned}
U_{l'} = \max \quad & \sum_{h \in H(l')} u_h x_{l'h} \\
\text{s.t.} \quad & \sum_{h \in H(l)} x_{lh} = 1, && \forall l \in L, \\
& \sum_{l \in C:\, h \in H(l)} x_{lh} \leq 1, && \forall d \in D,\ \forall C \in \mathcal{C}_d,\ \forall h \in H, \\
& x_{lh} \in \{0,1\}, && \forall l \in L,\ \forall h \in H(l).
\end{aligned}$$

This model keeps only the hard feasibility constraints and maximizes the capacity assigned to the single lecture l'. If $U_{l'}$ denotes its optimal value, then every hall $h \in H(l')$ with $u_h > U_{l'}$ may be removed from $H(l')$. Indeed, suppose that some globally feasible assignment uses lecture l' in a hall $\hat{h}$ with $u_{\hat{h}} > U_{l'}$. That same assignment satisfies the preprocessing constraints above and attains objective value at least $u_{\hat{h}}$, contradicting the definition of $U_{l'}$. Therefore the reduced set

$$\widehat{H}(l') = \{h \in H(l') : u_h \leq U_{l'}\}$$

preserves all feasible solutions of the original problem. Applying this optimization once to every lecture yields one valid global reduction pass for the MIQP, the compact MIP, and the CP model alike. Moreover, the pass can be repeated on the reduced compatibility sets. Since each pass preserves the feasible set of the preceding reduced model, and that reduced model already preserves the feasible set of the original problem, iterating the reduction until no compatibility set changes is still safe. In the nonlinear and compact MIP formulations, this directly removes assignment variables and all associated overlap and linking constraints. In the CP model, it shrinks the domains of the assignment variables and reduces the size of the distance tables or element encodings. In practice, the preprocessing used in our implementation is exactly this fixed-point procedure.

One may also use a cheaper lightweight version. Let

$$L'(l') = \{l'\} \cup \{l \in L : l \text{ overlaps } l' \text{ in time}\},$$

and solve the same maximization problem with $L'(l')$ replacing L and with the overlap constraints restricted accordingly. Let its optimal value be $\widetilde{U}_{l'}$. Since the lightweight problem removes lectures and therefore relaxes the feasible region, one has

$$\widetilde{U}_{l'} \geq U_{l'}.$$

Consequently, deleting from $H(l')$ only the halls with capacity larger than $\widetilde{U}_{l'}$ is still safe: it may leave more halls than the full preprocessing, but it cannot remove a hall that is needed by a globally feasible solution. As in the full version, one may apply the lightweight reduction repeatedly until a fixed point is reached. Each round remains safe for the same reason: it preserves the feasible region of the current reduced model, which in turn preserves that of the original problem.

3.5 Extensions with Additional Room Criteria

The base model isolates the walking-distance component of the lecture-hall assignment problem. In practical applications, one may wish to account for additional room-quality considerations without changing the main combinatorial structure.

These extensions also clarify the relation between the present model and the room-allocation approach of Antunes-Batista et al. (2026). Once feature requirements are incorporated into $H(l)$ and additional assignment penalties are allowed, the model moves closer to practical room-allocation formulations of the type studied there. However, the present framework remains distinct: student movement enters through a quadratic shared-enrollment walking term, capacity and overlap remain hard feasibility constraints, and the additional room-quality effects enter as linear assignment penalties within a single exact optimization model rather than as separate objectives in a multiobjective evolutionary framework.

First, hard room-feature requirements can be incorporated directly through the compatibility sets. Let F_l^{req} denote the set of mandatory features required by lecture l (for example, a laboratory session that must be assigned to a lab room), and let F_h denote the feature set of hall h. Then one may redefine

$$H(l) = \{h \in H : u_h \geq s_l,\ F_l^{\text{req}} \subseteq F_h\}.$$

No further change in the formulations is needed: the nonlinear model, the MILP reformulation, and the CP model already enforce assignment only to halls in $H(l)$.

Second, soft penalties can be added for room-feature mismatch and wasted capacity. Since both effects depend only on the assignment of lecture l to hall h, they may be combined into a single coefficient $p_{lh} \geq 0$, representing the total linear penalty associated with that assignment. For example, p_{lh} may aggregate penalties for missing desirable room features and for excess unused capacity under a chosen measure. The nonlinear objective (2) can then be extended to

$$\min \sum_{(l_1,l_2)\in A'} c_{l_1,l_2} \sum_{h_1\in H(l_1)} \sum_{h_2\in H(l_2)} d_{h_1,h_2}\, x_{l_1h_1} x_{l_2h_2} + \sum_{l\in L} \sum_{h\in H(l)} p_{lh} x_{lh}. \tag{18}$$

These additional terms are linear in the assignment variables, so they preserve the quadratic-plus-linear structure of the model. Consequently, the compact MILP reformulation is obtained simply by adding the same linear terms to (7), while keeping all linking constraints and valid inequalities unchanged. Likewise, in the CP model one may add per-lecture assignment costs indexed by the chosen hall. Thus these extensions move the model closer to practical room-allocation settings while preserving the exact-solution framework developed in this paper.

The timetabling literature also considers several side constraints that may be relevant for the quadratic room-assignment problem and can be accommodated by the three models above. In particular, the ITC 2019 competition model includes a *SameRoom* constraint requiring two classes, scheduled at non-overlapping times, to be assigned to the same room (Müller et al., 2025). This constraint can be formulated either as a hard constraint that must be satisfied by every feasible solution or as a soft constraint with an associated violation penalty. Let C_1^{hard} be the set of class pairs for which a hard SameRoom constraint is imposed. Then the following MIP and MIPQ constraints express it:

$$x_{l_1,h} = x_{l_2,h} \qquad \forall (l_1, l_2) \in C_1^{hard}, h \in H(l_1) \cap H(l_2), \tag{19}$$

and for the CP formulation it is even more straightforward:

$$a_{l_1} = a_{l_2} \qquad \forall (l_1, l_2) \in C_1^{hard} \tag{20}$$

In the hard case, the compatible room sets of the two classes can be reduced to the intersection of their original feasible sets, that is,

$$H'(l_1) = H'(l_2) = H(l_1) \cap H(l_2).$$

Alternatively, suppose that SameRoom is a soft constraint for a pair of classes l_1, l_2, with penalty π_{l_1,l_2} if the two classes are assigned to different rooms. Let C_1^{soft} be the set of such pairs, and for each pair introduce a binary variable v_{l_1,l_2} that equals 1 when the constraint is violated. A linear encoding for the MIP formulation is

$$v_{l_1,l_2} \geq x_{l_1,h} - x_{l_2,h} \qquad \forall (l_1, l_2) \in C_1^{soft},\ h \in H(l_1) \cap H(l_2), \tag{21}$$

$$v_{l_1,l_2} \geq x_{l_2,h} - x_{l_1,h} \qquad \forall (l_1, l_2) \in C_1^{soft},\ h \in H(l_1) \cap H(l_2), \tag{22}$$

$$v_{l_1,l_2} \geq x_{l_1,h} \qquad \forall (l_1, l_2) \in C_1^{soft},\ h \in H(l_1) \setminus H(l_2), \tag{23}$$

$$v_{l_1,l_2} \geq x_{l_2,h} \qquad \forall (l_1, l_2) \in C_1^{soft},\ h \in H(l_2) \setminus H(l_1). \tag{24}$$

The corresponding penalty term

$$\sum_{(l_1,l_2) \in C_1^{soft}} \pi_{l_1,l_2} v_{l_1,l_2}$$

is then added to the objective function. In the nonlinear MIPQ formulation, one may avoid the auxiliary variables entirely and add the direct bilinear penalty

$$\sum_{(l_1,l_2) \in C_1^{soft}} \pi_{l_1,l_2} \left(1 - \sum_{h \in H(l_1) \cap H(l_2)} x_{l_1 h} x_{l_2 h} \right).$$

Because each lecture is assigned to exactly one hall, the inner sum equals 1 if and only if the two lectures use the same compatible hall, and equals 0 otherwise, so this term is exactly the SameRoom violation penalty. Implementation of the soft SameRoom constraint in the CP model is more straightforward: one may simply add the term

$$\sum_{(l_1,l_2)\in C_1^{soft}} \pi_{l_1,l_2} \left[a_{l_1} \neq a_{l_2}\right]$$

using a reification construct that evaluates to 1 when the logical condition is true and to 0 otherwise.

An important caveat is that SameRoom constraints, in both their hard and soft forms, are typically defined over the full weekly timetable. For example, a class given on Monday may be required to use the same room as a related class given on Wednesday. If such cross-day constraints are enforced, the problem is no longer decomposable by day, and one must solve at least the whole week, and possibly the whole term, simultaneously.

Another common timetabling side constraint that is also relevant here, albeit in a more restricted sense, is *SameAttendees* (Müller et al., 2025). In the timetabling model, this constraint limits the travel time between two classes that may share students. In our setting, where event times are fixed in advance, it becomes a restriction on the admissible room pairs for two given classes. Let C_2^{hard} be the set of class pairs with a hard SameAttendees constraint, and let

$$F_{l_1,l_2} \subseteq H(l_1) \times H(l_2)$$

denote the set of forbidden room pairs for a given pair (l_1, l_2). If l_1 ends before l_2 starts, then $(h_1, h_2) \in F_{l_1,l_2}$ whenever

$$d_{h_1,h_2} > \text{start}(l_2) - \text{end}(l_1).$$

If l_2 ends before l_1 starts, then $(h_1, h_2) \in F_{l_1,l_2}$ whenever

$$d_{h_2,h_1} > \text{start}(l_1) - \text{end}(l_2).$$

If the two classes overlap in time, then every room pair is forbidden. The hard constraint is therefore expressed in the MIP and MIPQ models by

$$x_{l_1,h_1} + x_{l_2,h_2} \leq 1 \qquad \forall (l_1, l_2) \in C_2^{hard},\ (h_1, h_2) \in F_{l_1,l_2}. \tag{25}$$

In the CP model, one may equivalently forbid the assignments in F_{l_1,l_2} directly.

Now let C_2^{soft} be the set of soft SameAttendees pairs, with penalty ρ_{l_1,l_2} for violating pair (l_1, l_2). Introduce a binary variable u_{l_1,l_2} that equals 1 when the chosen room pair belongs to F_{l_1,l_2}. Then the MIP model can enforce

$$u_{l_1,l_2} \geq x_{l_1,h_1} + x_{l_2,h_2} - 1 \qquad \forall (l_1, l_2) \in C_2^{soft},\ (h_1, h_2) \in F_{l_1,l_2}, \tag{26}$$

and add the penalty term

$$\sum_{(l_1,l_2)\in C_2^{soft}} \rho_{l_1,l_2} u_{l_1,l_2}$$

to the objective. In the nonlinear MIPQ formulation, the same penalty can be written directly as

$$\sum_{(l_1,l_2)\in C_2^{soft}} \rho_{l_1,l_2} \sum_{(h_1,h_2)\in F_{l_1,l_2}} x_{l_1,h_1} x_{l_2,h_2}.$$

Because exactly one room pair is selected for (l_1, l_2), the inner sum is a binary indicator of whether the chosen pair belongs to F_{l_1,l_2}. This avoids both auxiliary variables and the accompanying linking inequalities in the MIPQ model. In the CP model, the same logic can be implemented by a reified table constraint that links the chosen room pair to a binary violation indicator.

In the MIP model, the pairwise formulations above can be strengthened using another set biclique constraints. Let $S_1 \subseteq H(l_1)$ and $S_2 \subseteq H(l_2)$ be such that

$$S_1 \times S_2 \subseteq F_{l_1,l_2}.$$

Then every assignment that places l_1 in a room from S and l_2 in a room from T necessarily violates SameAttendees. Hence the following aggregated inequalities are valid:

$$\sum_{h_1 \in S_1} x_{l_1,h_1} + \sum_{h_2 \in S_2} x_{l_2,h_2} \leq 1 \qquad \forall (l_1, l_2) \in C_2^{hard},\ S_1 \times S_2 \subseteq F_{l_1,l_2}, \tag{27}$$

and

$$u_{l_1,l_2} \geq \sum_{h_1 \in S_1} x_{l_1,h_1} + \sum_{h_2 \in S_2} x_{l_2,h_2} - 1 \qquad \forall (l_1, l_2) \in C_2^{soft},\ S_1 \times S_2 \subseteq F_{l_1,l_2}. \tag{28}$$

These inequalities dominate the basic pairwise constraints whenever $|S_1| > 1$ or $|S_2| > 1$. In the implementation, the extended cut mode constructs such sets by starting from one anchor room of l_1, collecting all rooms of l_2 that are forbidden with that anchor, and then enlarging the l_1-side set to every room that is incompatible with the entire resulting set on the l_2 side. This is directly analogous in spirit to the extended distance cuts used for the quadratic walking term. These aggregated inequalities are useful for the linearized MIP and for the CP reified encoding; in the nonlinear MIPQ formulation, the soft SameAttendees penalties are represented directly by bilinear objective terms instead.

Hard SameRoom and SameAttendees constraints can also be incorporated into the compatibility preprocessing step described in Section 3.4.

4 Numerical experiments

In this section we evaluate the computational behavior of the proposed formulations and algorithms and quantify the contribution of each modeling enhancement.

The numerical study address four main questions: (i) how much stronger the compact MIP reformulation is than the baseline nonlinear model, (ii) how much the valid inequalities improve the linear relaxation and the branch-and-bound performance, (iii) how effective the preprocessing procedures are in shrinking the compatibility sets and accelerating solution, and (iv) how the MIP and CP approaches compare on instances of various sizes and densities.

4.1 Test instances

Our real-world test set is derived from two XML data sources: one instance from the ITC 2019 university course timetabling benchmark (Müller et al., 2025) and a 2023 institutional dataset

from the University of Lancaster that was kindly provided to us by Matthew Davison, the first author of Davison et al. (2025). In both sources, the original room set, room capacities, and room-to-room distances are retained, and the weekly timetabling data are transformed into daily lecture-hall assignment instances because the optimization model is separable by day whenever no cross-day side constraints are enforced. The resulting benchmark therefore preserves realistic room heterogeneity and authentic side-constraint structure while isolating the daily room-assignment subproblem studied in this paper.

The ITC 2019 repository contains 36 real-world timetabling instances in total, namely 30 competition benchmarks and 6 test instances (Müller et al., 2025). We use instance from Purdue, spring term of 2007 (denoted in the repository as `pu-c8-spr07`). The instance belongs to the test set and is posted together with the best timetable available in the ITC2019 contest repository, see `https://www.itc2019.org/`. Among the instances that provide explicit room-to-room walking distances, the Purdue instance is the largest, which is why we selected it for the present study. The solution file used in our experiments is attributed in the XML metadata to Tomáš Müller, and we therefore use the repository solution submitted under his name as the underlying timetable from which room assignments and student flows are extracted. Since the ITC data describe a full semester in which the exact weekly timetable may vary over time, our instance-construction procedure selects the most loaded teaching week, where load is measured by the number of active room-requiring classes in the published timetable. For `pu-c8-spr07`, this rule selects week 2. The selected week is then decomposed into one instance per active day. For each day, the data-transformation pipeline keeps the original room-assignment penalties from the ITC XML, extracts the hard and soft *SameRoom* pairs and the hard *SameAttendees* pairs, and constructs the successor relation from the student-course records using an automatically inferred short-break threshold. The current implementation also applies the default ITC capacity correction whenever the published timetable places a class in a room smaller than the reported class size; in that case the class size is reduced to the assigned room capacity before compatibility sets are built. This produces five daily ITC instances, all with the same 213 halls, with between 726 and 953 lectures per day and between 698 and 3,352 positive successor pairs.

For the Lancaster instances, we use the actual timetable and anonymized student registration data used at the University of Lancaster in 2023, which we obtained directly from Matthew Davison. We gratefully acknowledge Matthew Davison for sharing these data with us. Here the construction is less direct because the institutional data contain weekly course configurations and also *SameClass* groups. In the XML semantics, a *SameClass* relation indicates that several class identifiers are administrative representations of one and the same teaching activity and are therefore required to share exactly the same scheduled time and room. In the Lancaster 2023 data, such duplicated representations arise from institutional formalities, so our preprocessing pipeline contracts each *SameClass* component into one representative activity before any room-assignment instance is generated. This consolidation is essential to avoid artificial duplication of decision variables for what is, operationally, a single lecture.

After this consolidation step, the Lancaster pipeline identifies the two main teaching terms and, within each term, selects the most loaded teaching week, where load is measured by the number of active room-requiring merged activities. In the present data this rule selects week 13 for term 1 and week 28 for term 2. Student registrations are then projected onto the selected week and repaired to obtain a consistent flow-based room-assignment instance. Two small adjustments are required. First, the merged weekly timetable is used to eliminate formal duplicate activities created by the

SameClass representation just discussed. Second, a small number of students are registered to more than one class at the same time; This is an artifact of university formality that require registering a student to a class that they do not physcally attend just to enable theie access to the course online material or to allow them attend the final exam . Because such simultaneous registrations do not induce a meaningful movement pattern between rooms, and because the quadratic objective relies precisely on a well-defined flow of students between consecutive lectures, these conflicting registrations are broken by removing the student from one of the overlapping classes. The cleaned weekly registrations are then mapped to daily lecture records, and the hard and soft side constraints are inherited from the original XML after mapping through the merged components. In the resulting Lancaster instances there are hard and soft *SameAttendees* constraints but no remaining *SameRoom* constraints. This yields ten daily Lancaster instances, all with 660 halls, with between 243 and 510 lectures per day and between 117 and 389 positive successor pairs.

Overall, the real-world benchmark family used in the numerical study contains 15 daily instances: five Purdue spring 2007 days and ten Lancaster days. The two sources are complementary. The Purdue instance is tighter in its timetable-driven side constraints and much denser in immediate-successor structure, whereas the Lancaster instances are built from a much larger hall set and from cleaned weekly registration data that are projected into a flow network of consecutive student movements. Together they provide a useful test bed for assessing the robustness of the MIPQ, MIP, and CP formulations across materially different academic data-generation pipelines.

It should be noted that while both datset were previously used as benchmarks instances for the university course timetabeling problem that includes among other decision assignment of classes to rooms, the result from the literature are not comparable because none of the previous studies consider an objective that includes the quadratic assiggbment component, i.e., minimizing the total walking distances of the students. While, we can calculate the walking distance of the students from the solutions psted by previus studies, it would not be fare to compare our optimal solutions to those obtained by algorithms that optimized a different objective function.

4.2 Compared formulations and algorithmic variants

The comparison should include at least the following variants:

1. the baseline nonlinear formulation;
2. the compact MIP reformulation without strengthening;
3. the compact MIP reformulation with valid inequalities;
4. the compact MIP reformulation with valid inequalities and preprocessing;
5. the CP formulation;
6. optionally, the extended model with additional linear assignment penalties.

If both lightweight and full preprocessing are retained, they should be tested as separate variants.

4.3 Performance measures

For each run, the numerical study should report the objective value, runtime, optimality gap at the time limit, and a solved-to-optimality indicator. For MIP-based variants, it is also useful to report the root relaxation bound, the number of branch-and-bound nodes, and the reduction in compatibility-set size produced by preprocessing. Aggregate statistics should include means, medians, and success rates over the seed set.

4.4 Experiment blocks

The numerical section can be organized into the following blocks. The first block should compare the baseline nonlinear model against the compact MIP reformulation to isolate the value of linearization. The second block should quantify the marginal benefit of the strengthening inequalities. The third block should evaluate lightweight versus full preprocessing. The fourth block should compare the best MIP variant with the CP formulation. A final block may analyze scalability by plotting runtime and solution quality against instance size and density.

4.5 Implementation and reproducibility

This subsection should document the hardware, solver versions, parameter settings, and time limits used in the experiments. It should also state the random-seed policy and the exact generator settings so that the experiments can be reproduced. If the experiments rely on the current code base, the subsection may describe the implemented generator, the preprocessing options, and the solver configurations used for each formulation.

References

Sina Abdipoor, Razali Yaakob, Say Leng Goh, and Salwani Abdullah. Meta-heuristic approaches for the university course timetabling problem. *Intelligent Systems with Applications*, 19:200253, 2023. doi: 10.1016/j.iswa.2023.200253.

Warren P. Adams and Terri A. Johnson. Improved linear programming-based lower bounds for the quadratic assignment problem. In *Quadratic Assignment and Related Problems*, volume 16 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 43–75. American Mathematical Society, 1994.

Kurt Anstreicher, Nathan Brixius, Jean-Pierre Goux, and Jeff Linderoth. Solving large quadratic assignment problems on computational grids. *Mathematical Programming*, 91(3):563–588, 2002. doi: 10.1007/s101070100260.

Kurt M. Anstreicher. Recent advances in the solution of quadratic assignment problems. *Mathematical Programming*, 97(1–2):27–42, 2003. doi: 10.1007/s10107-003-0437-z.

Bruno Antunes-Batista, Soumen Atta, Vítor Basto-Fernandes, Iryna Yevseyeva, and Michael T. M. Emmerich. Multiobjective optimization approaches for room allocation in university course timetabling. *Procedia Computer Science*, 278:663–670, 2026. doi: 10.1016/j.procs.2026.03.037.

Rainer E. Burkard, Eranda Çela, Panos M. Pardalos, and Leonidas S. Pitsoulis. The quadratic assignment problem. In *Handbook of Combinatorial Optimization*, pages 241–338. Kluwer Academic Publishers, 1998.

Edmund K. Burke and Sanja Petrovic. Recent research directions in automated timetabling. *European Journal of Operational Research*, 140(2):266–280, 2002. doi: 10.1016/S0377-2217(02)00069-3.

Michael W. Carter and Craig A. Tovey. When is the classroom assignment problem hard? *Operations Research*, 40(1-Supplement-1):S28–S39, 1992. doi: 10.1287/opre.40.1.S28.

Matthew Davison, Ahmed Kheiri, and Konstantinos G. Zografos. Modelling and solving the university course timetabling problem with hybrid teaching considerations. *Journal of Scheduling*, 28(2):195–215, 2025. doi: 10.1007/s10951-024-00817-w.

A. M. Frieze and J. Yadegar. On the quadratic assignment problem. *Discrete Applied Mathematics*, 5(1):89–98, 1983. doi: 10.1016/0166-218X(83)90018-5.

P. C. Gilmore. Optimal and suboptimal algorithms for the quadratic assignment problem. *Journal of the Society for Industrial and Applied Mathematics*, 10(2):305–313, 1962. doi: 10.1137/0110022.

L. Kaufman and F. Broeckx. An algorithm for the quadratic assignment problem using Bender's decomposition. *European Journal of Operational Research*, 2(3):207–211, 1978. doi: 10.1016/0377-2217(78)90095-4.

Antoon W. J. Kolen, Jan Karel Lenstra, Christos H. Papadimitriou, and Frits C. R. Spieksma. Interval scheduling: A survey. *Naval Research Logistics*, 54(5):530–543, 2007. doi: 10.1002/nav.20231.

Tjalling C. Koopmans and Martin Beckmann. Assignment problems and the location of economic activities. *Econometrica*, 25(1):53–76, 1957. doi: 10.2307/1907742.

Eugene L. Lawler. The quadratic assignment problem. *Management Science*, 9(4):586–599, 1963. doi: 10.1287/mnsc.9.4.586.

Eliane M. Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter Hahn, and Tania Querido. A survey for the quadratic assignment problem. *European Journal of Operational Research*, 176(2):657–690, 2007. doi: 10.1016/j.ejor.2005.09.032.

Tomáš Müller, Hana Rudová, and Zuzana Müllerová. Real-world university course timetabling at the international timetabling competition 2019. *Journal of Scheduling*, 28(2):247–267, 2025. doi: 10.1007/s10951-023-00801-w.

Efstratios Rappos, Eric Thi'emard, Stephan Robert, and Jean-François Hêche. A mixed-integer programming approach for solving university course timetabling problems. *Journal of Scheduling*, 25(4):391–404, 2022. doi: 10.1007/s10951-021-00715-5.

Sartaj Sahni and Teofilo Gonzalez. P-complete approximation problems. *Journal of the ACM (JACM)*, 23(3):555–565, 1976.

Andrea Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999. doi: 10.1023/A:1006576209967.

Éric Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17 (4–5):443–455, 1991. doi: 10.1016/S0167-8191(05)80147-4.

Camilo Torres-Ovalle, Jairo R. Montoya-Torres, Carlos Quintero-Araujo, and Angelica Sarmiento. University course scheduling and classroom assignment. *Ingeniería y Universidad*, 18(1):59–75, 2014. doi: 10.11144/Javeriana.IYU18-1.ucsc.

Qing Zhao, Stefan E. Karisch, Franz Rendl, and Henry Wolkowicz. Semidefinite programming relaxations for the quadratic assignment problem. *Journal of Combinatorial Optimization*, 2(1): 71–109, 1998. doi: 10.1023/A:1009795911987.

A Synthetic instance generator

The numerical study relies on a synthetic instance generator implemented in the accompanying code base. The purpose of this generator is not to reproduce one specific university in all operational details, but rather to create controlled benchmark instances that preserve the structural drivers of the lecture-hall assignment problem: heterogeneous room sizes, fixed-time overlap patterns, cohort-based demand, sparse but meaningful successor relations, and compatibility sets that are neither trivial nor artificially restrictive. Since the problem is separable by day, every generated instance represents a single teaching day. This keeps the benchmark focused on the intrinsic daily assignment difficulty without any loss of generality relative to a weekly formulation.

The generator first creates the halls. Each hall is assigned a capacity drawn from a broad range and a location in a two-dimensional campus plane. Inter-hall distances are then obtained from scaled Euclidean distances and rounded upward to integers. This construction reflects two practical considerations. First, universities typically operate a heterogeneous stock of classrooms and lecture halls, so uniform room capacities would understate the assignment difficulty. Second, a geometric distance model yields a coherent metric structure, which is more realistic than an arbitrary distance matrix and better aligned with the interpretation of the objective as walking effort across campus.

Next, the generator creates a fixed lecture timetable. Let $\delta \in (0, 1]$ denote the target density, interpreted as the fraction of available hall-time slots that should be occupied by lectures. The total occupied volume is therefore chosen as approximately $\delta|H||T|$, where $|T|$ is the number of time slots in the single day. This occupied volume is decomposed into lecture durations of 2, 3, or 4 consecutive slots and then packed into hall-day bins. Within each bin, random idle gaps are inserted between lectures. The use of a density parameter makes the benchmark scalable in a transparent way: increasing δ simultaneously increases room contention and the size of the overlap graph. Restricting durations to a small range of moderate lengths is also practically reasonable, since most university teaching activities are neither one-slot micro-events nor all-day blocks.

Each generated lecture is then assigned an academic identity consisting of a subject, a study year, and a compulsory/elective flag. In the current implementation, subjects are balanced across a fixed list of disciplines, study years are balanced across four cohorts, and approximately 70% of lectures

are designated compulsory. The balance requirement prevents degenerate instances in which one subject or one year dominates the benchmark purely by chance. More importantly, the generator imposes a cohort-overlap rule: for any fixed subject-year cohort and time slot, there may be at most one compulsory lecture, or at most two elective lectures, but not both simultaneously. This rule encodes a common curricular reality. Compulsory offerings for a cohort are usually scheduled so as not to conflict with each other, whereas a small amount of parallelism among electives is realistic and even desirable.

The implementation contains a two-level fallback mechanism to enforce these curriculum rules robustly on dense instances. First, the generator computes the maximum simultaneous lecture volume implied by the cohort system itself. With the current choice of eight subjects and four study years, there are 32 cohorts, so the overlap rule permits at most 64 active lectures in any slot. If the requested density would force more lecture-slots than this concurrency cap allows, the instance request is declared infeasible immediately rather than being passed to a downstream routine that would inevitably fail. Second, even when the requested density is globally feasible, a particular random packing of lecture durations into the day may still create an overly congested slot pattern or a timetable for which the greedy attribute assignment gets stuck. In that case the code first tries a small exact CP-SAT feasibility model for the balanced attribute-assignment subproblem. If that still fails, the timetable layout itself is redrawn and the process is repeated. This safeguard has a methodological purpose: hard benchmark instances should arise from the optimization problem, not from accidental inconsistencies in the synthetic curriculum generator.

After the timetable structure is fixed, the generator simulates student demand rather than drawing lecture sizes independently. This choice is important because the objective depends on shared enrollments between consecutive lectures, so generating only lecture sizes would miss the main source of quadratic coupling. The code first estimates the size of each subject-year cohort from the capacities of its compulsory offerings. Intuitively, compulsory lectures anchor the scale of a cohort better than electives do, because they are the courses that most students in the cohort are expected to attend. The generator then simulates individual student journeys through the day. Students almost always attend compulsory lectures of their own cohort, attend home-cohort electives with moderate probability, only occasionally attend previous-year electives in the same subject, and only very rarely attend electives from another subject. When several candidate electives are available, the choice is weighted by a lecture-specific popularity factor and by the remaining hidden room capacity. This produces three desirable effects simultaneously: large core classes emerge naturally, elective demand is split across parallels, and successor weights are concentrated on academically plausible lecture pairs rather than spread uniformly across the timetable.

The generator also introduces a latent or hidden hall for each lecture during the data-creation stage. This hidden hall is used only to shape the simulated demand and is not revealed to the optimizer as a prescribed assignment. After student journeys are simulated, lecture attendance is tightened upward toward the capacity of the hidden hall. Compulsory lectures are pushed to a higher occupancy band than electives, and lecture sizes are also kept above the next smaller capacity tier whenever possible. The practical motivation is to avoid synthetic instances in which most lectures are far smaller than the rooms that can host them. Without this tightening step, the compatibility sets would often be excessively large, the capacity constraints would be weak, and the wasted-space penalty would rarely matter. The tightening procedure therefore preserves feasibility while making room choice consequential, which is exactly the regime in which lecture-hall assignment is operationally interesting.

Finally, the benchmark data used by the optimization models are derived from the simulated attendance. For each lecture l, the compatibility set $H(l)$ contains all halls whose capacity is at least the final attendance of l. For each pair of lectures (l_1, l_2), the common-student weight c_{l_1,l_2} is defined by the number of simulated students who attend l_1 and then l_2 in immediately consecutive time slots on the same day. This ensures that the successor relation is sparse and behaviorally grounded: only lecture pairs that arise naturally from student schedules receive positive walking weight. In the extended objective used in the code, each compatible assignment (l, h) also receives a penalty for severe under-utilization of the hall. The penalty is zero as long as at least 90% of the hall is filled, and it then increases quadratically with the number of excess empty seats beyond that threshold. This reflects the practical view that assigning a class to a slightly oversized room may be acceptable, whereas assigning it to a much larger room becomes increasingly undesirable.